

Exploring Managed Execution in C#



Mike Woodring

Programmer | Learner | Teacher

@mcwoodring | [linkedin.com/in/woodring](https://www.linkedin.com/in/woodring)



Code Execution Models

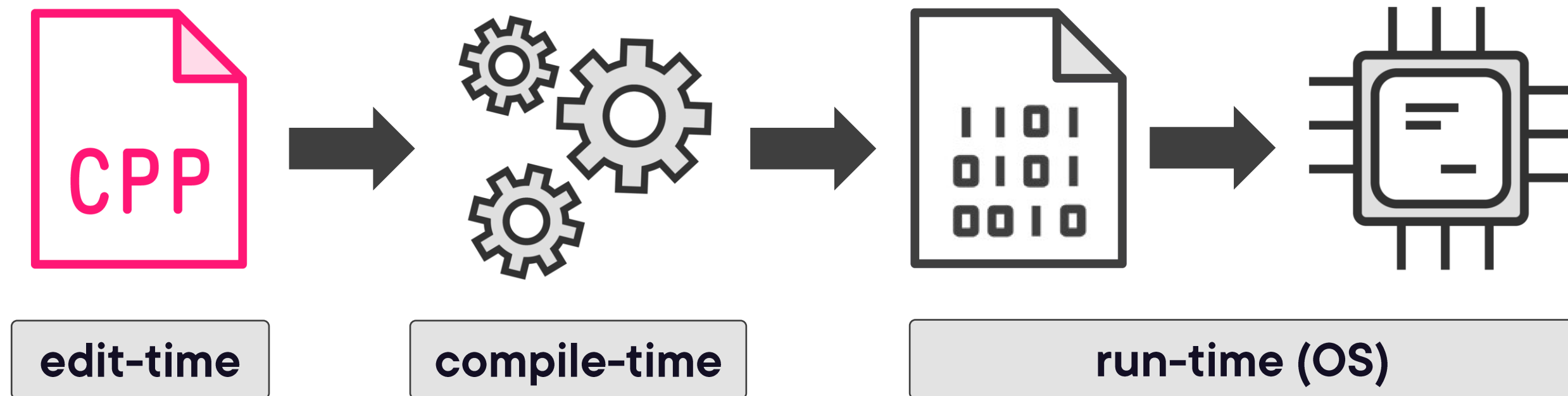
Compiled | Native

Strong/static typing

Compile-time type safety

Manual memory management

Fast(est) performance profile



Code Execution Models

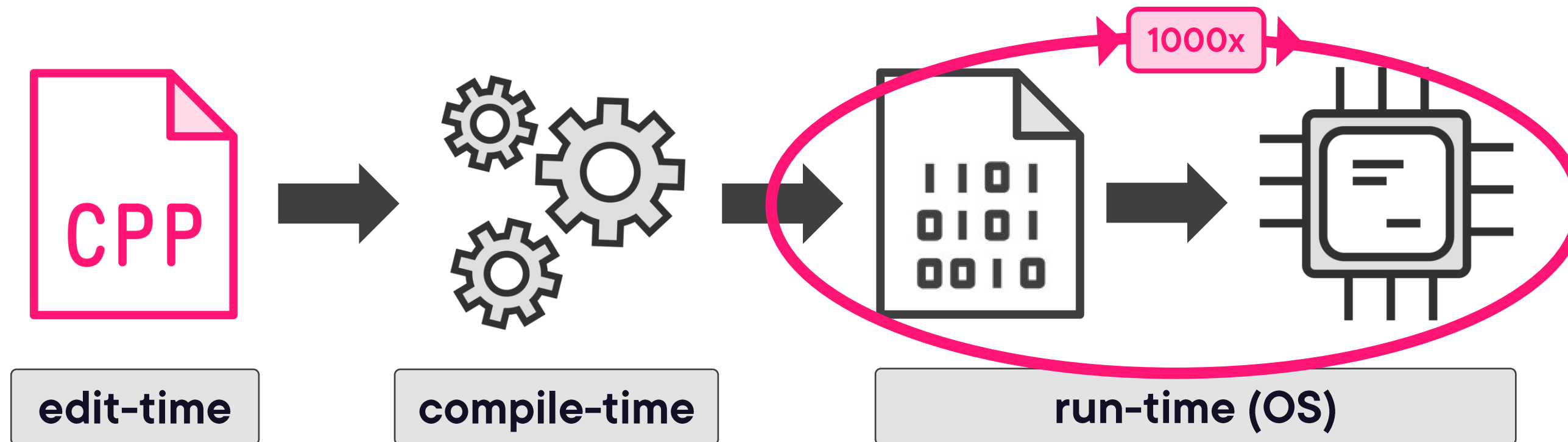
Compiled | Native

Strong/static typing

Compile-time type safety

Manual memory management

Fast(est) performance profile



Code Execution Models

Compiled | Native

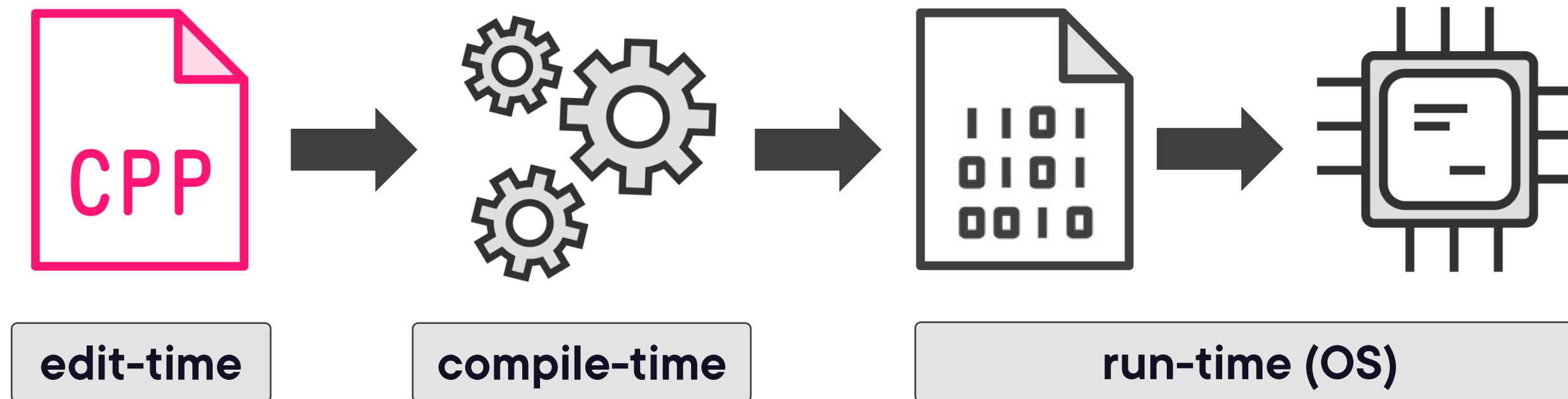
Strong/static typing

Compile-time type safety

Manual memory management

Fast(est) performance profile

- ✗ **brittle – poor resilience to change**
- ✗ **cross-compilation necessary for portability**
- ✗ **error prone memory management**



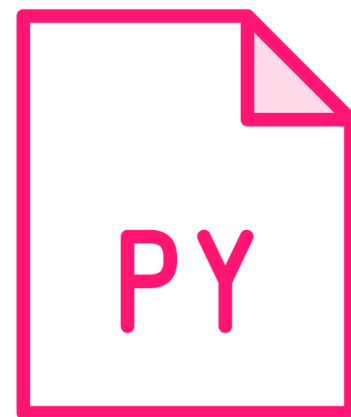
Code Execution Models

Compiled | Native

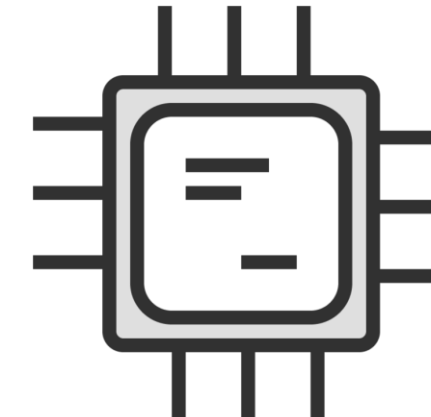
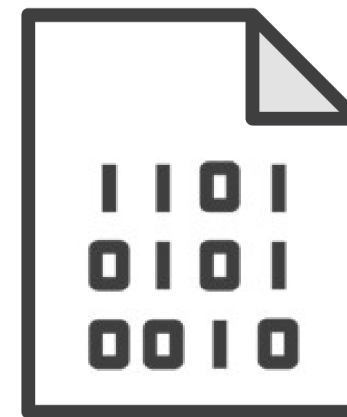
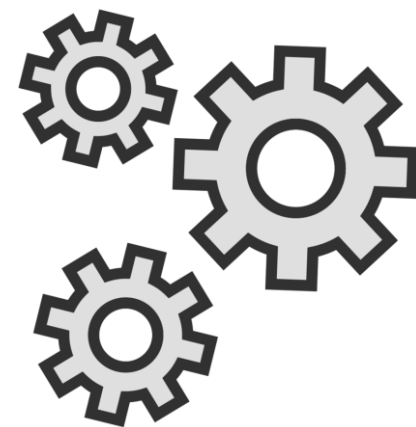
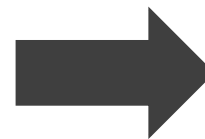
- Strong/static typing
- Compile-time type safety
- Manual memory management
- Fast(est) performance profile

Interpreted | Dynamic (REPL)

- Loose/dynamic typing
- Permissive runtime type conversion
- Automatic memory management
- Slow performance profile



edit-time



run-time (interpreter)



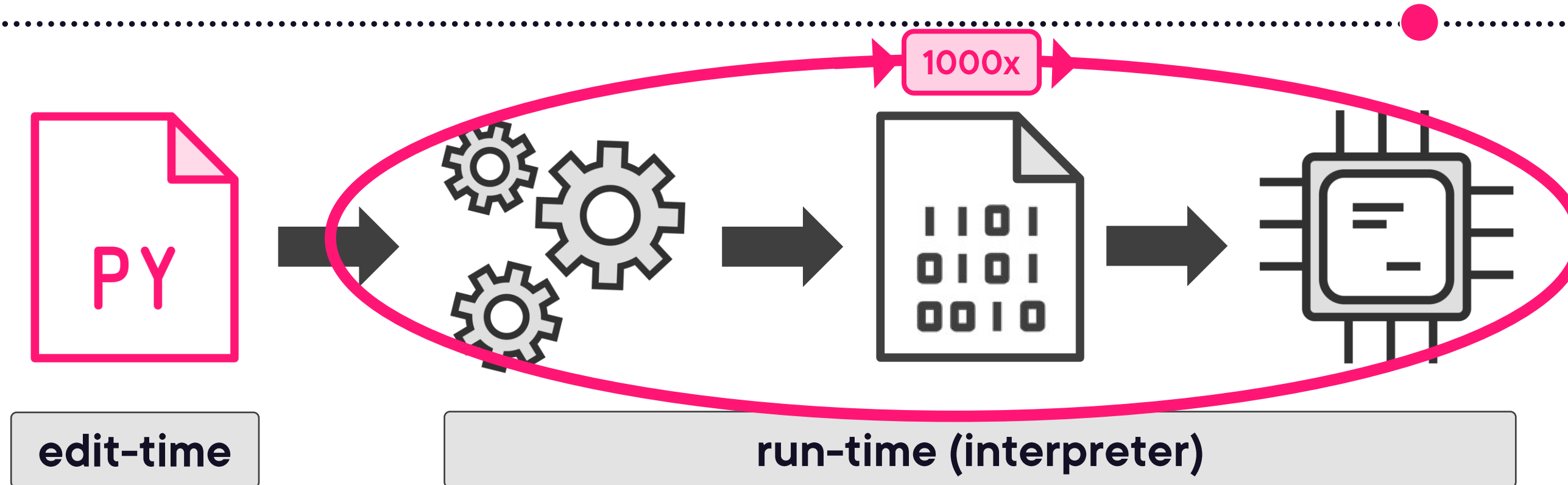
Code Execution Models

Compiled | Native

- Strong/static typing
- Compile-time type safety
- Manual memory management
- Fast(est) performance profile

Interpreted | Dynamic (REPL)

- Loose/dynamic typing
- Permissive runtime type conversion
- Automatic memory management
- Slow performance profile



Code Execution Models

Compiled | Native

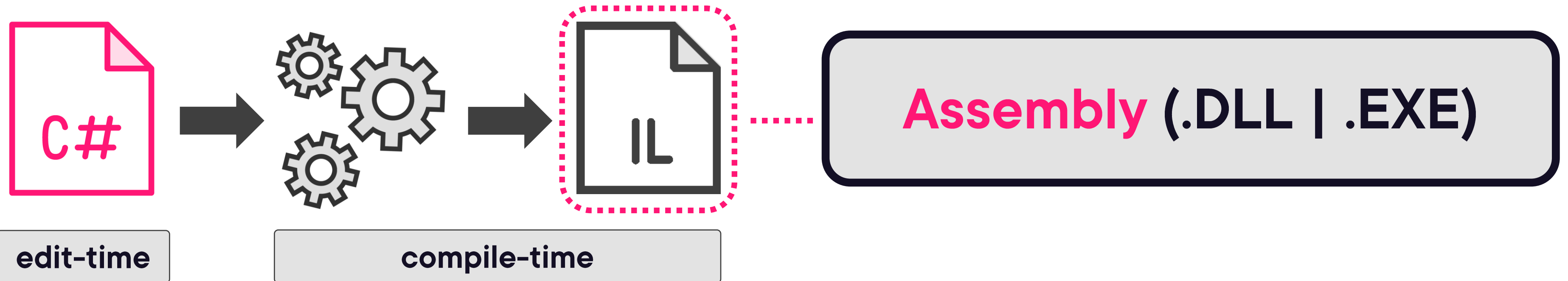
- Strong/static typing
- Compile-time type safety
- Manual memory management
- Fast(est) performance profile

Managed | Execution Engine

- Strong typing
- Runtime type safety
- Garbage collection
- Native code performance

Interpreted | Dynamic (REPL)

- Loose/dynamic typing
- Permissive runtime type conversion
- Automatic memory management
- Slow performance profile



Code Execution Models

Compiled | Native

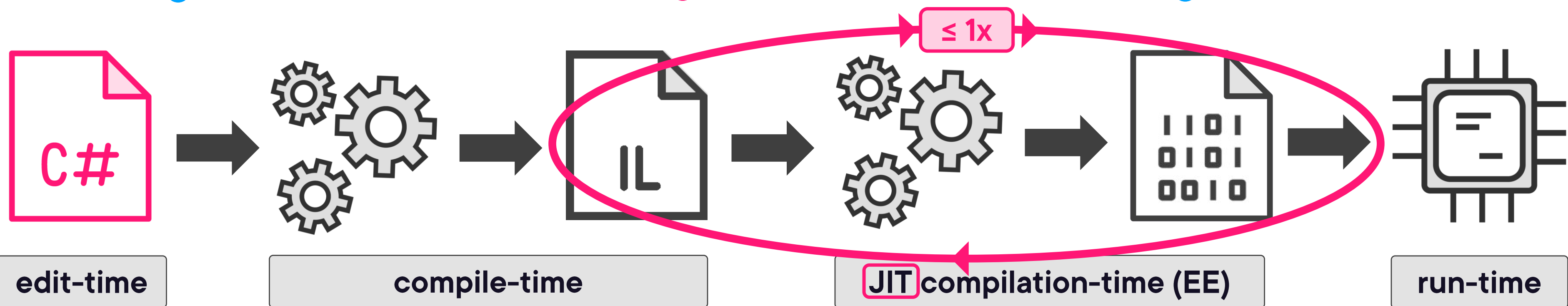
- Strong/static typing
- Compile-time type safety
- Manual memory management
- Fast(est) performance profile

Managed | Execution Engine

- Strong typing
- Runtime type safety
- Garbage collection
- Native code performance

Interpreted | Dynamic (REPL)

- Loose/dynamic typing
- Permissive runtime type conversion
- Automatic memory management
- Slow performance profile



Code Execution Models

Compiled | Native

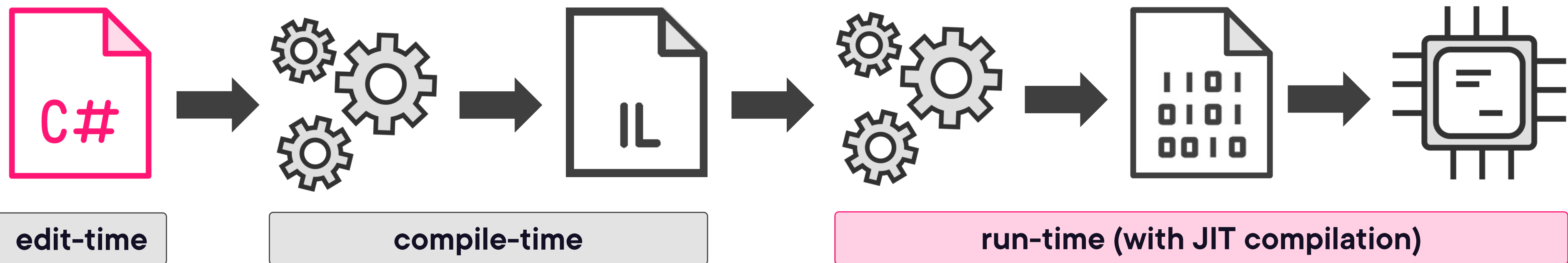
- Strong/static typing
- Compile-time type safety
- Manual memory management
- Fast(est) performance profile

Managed | Execution Engine

- Strong typing
- Runtime type safety
- Garbage collection
- Native code performance

Interpreted | Dynamic (REPL)

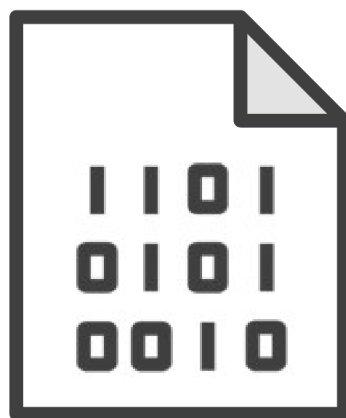
- Loose/dynamic typing
- Permissive runtime type conversion
- Automatic memory management
- Slow performance profile



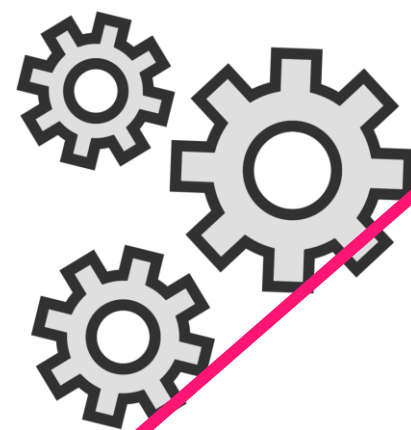
JIT Compilation – Method Never Called

```
static void Main()  
{  
    Console.WriteLine(42);  
}
```

```
static int Add(int a, int b)  
{  
    return (a + b);  
}
```



JIT-compiled code



JIT compiler (EE)



method (not) being called

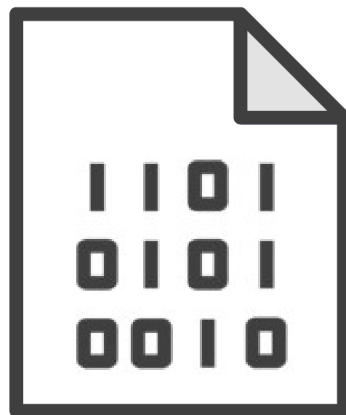


JIT Compilation – Method Called (Before)

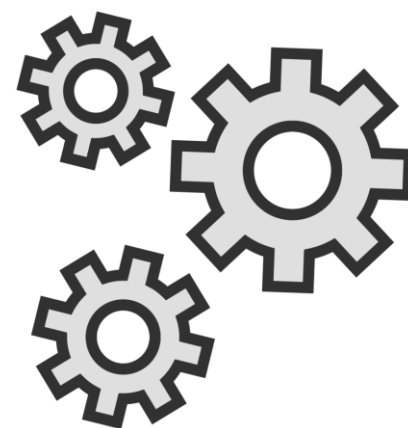
```
static void Main()  
{  
    var sum = Add(30, 12);  
    Console.WriteLine(sum);  
}
```



```
static int Add(int a, int b)  
{  
    return (a + b);  
}
```



JIT-compiled code



JIT compiler (EE)



method being called

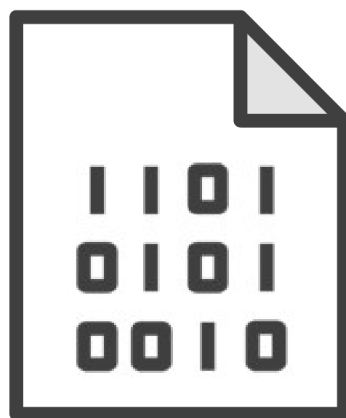


JIT Compilation – Method Called (After)

```
static void Main()  
{  
    var sum = Add(30, 12);  
    Console.WriteLine(sum);  
}
```



```
static int Add(int a, int b)  
{  
    return (a + b);  
}
```



JIT-compiled code



method being called



Code Execution Models

Compiled | Native

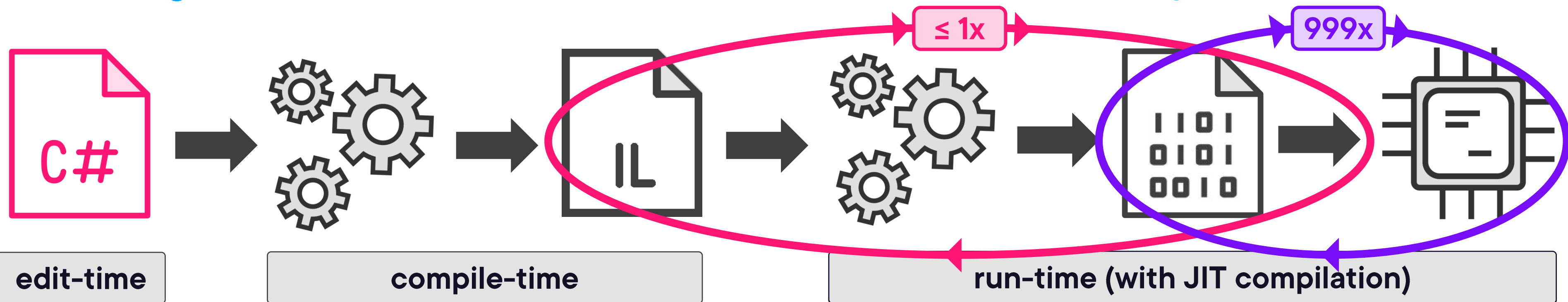
- Strong/static typing
- Compile-time type safety
- Manual memory management
- Fast(est) performance profile

Managed | Execution Engine

- Strong typing
- Runtime type safety
- Garbage collection
- Native code performance

Interpreted | Dynamic (REPL)

- Loose/dynamic typing
- Permissive runtime type conversion
- Automatic memory management
- Slow performance profile



Code Execution Models

Compiled | Native

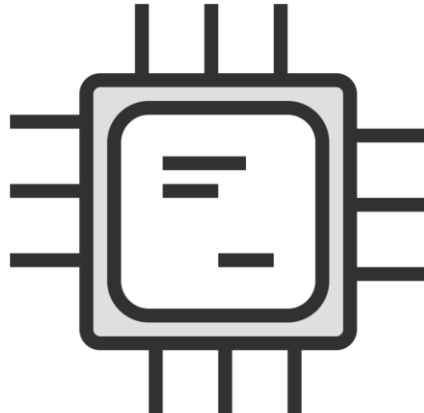
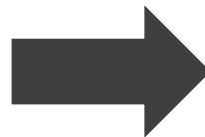
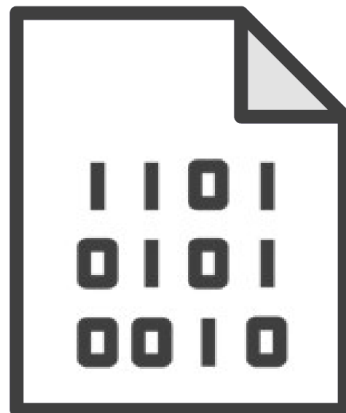
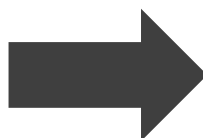
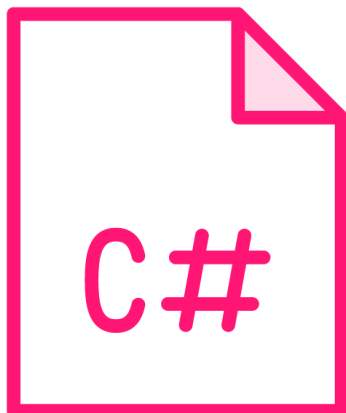
- Strong/static typing
- Compile-time type safety
- Manual memory management
- Fast(est) performance profile

Managed | Execution Engine

- Strong typing
- Runtime type safety
- Garbage collection
- Native code performance

Interpreted | Dynamic (REPL)

- Loose/dynamic typing
- Permissive runtime type conversion
- Automatic memory management
- Slow performance profile



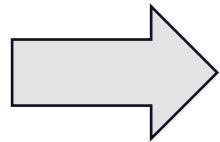
edit-time

compile-time

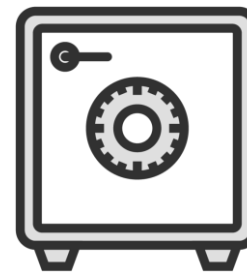
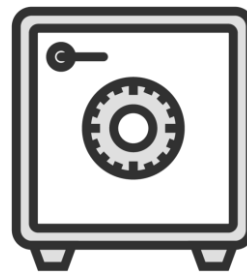
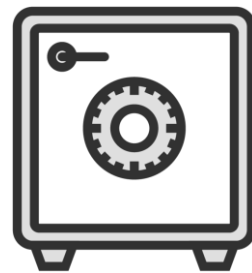
run-time (with JIT compilation)



Execution Engine



Just-in-Time (JIT) Compilation



Runtime Type Safety



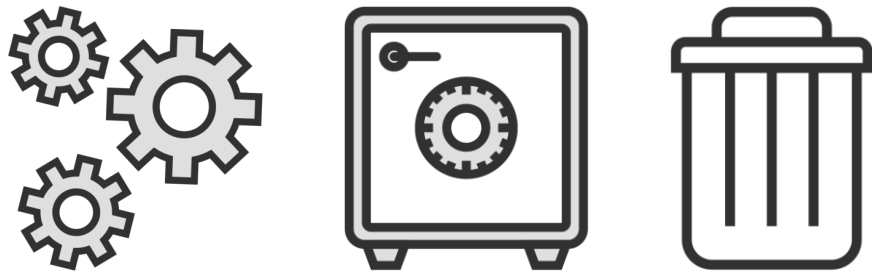
Garbage Collection (GC)

Common Language Runtime (CLR)

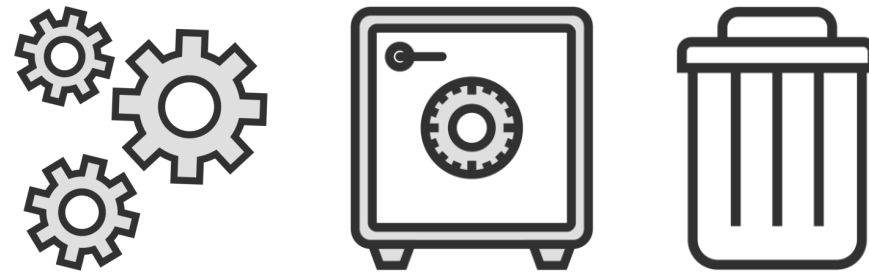


Common Language Runtimes

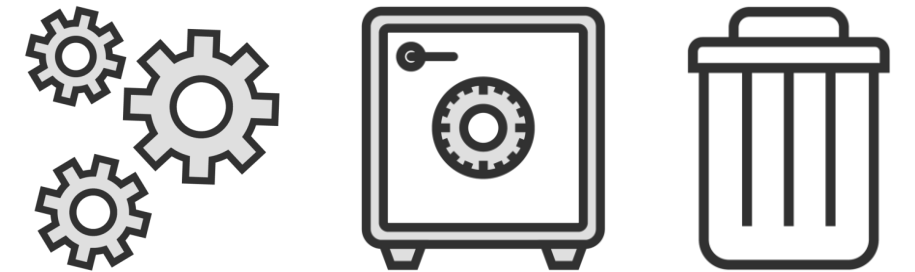
.NET 8



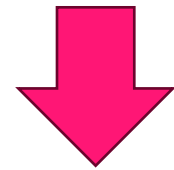
.NET Core | Mono | ...



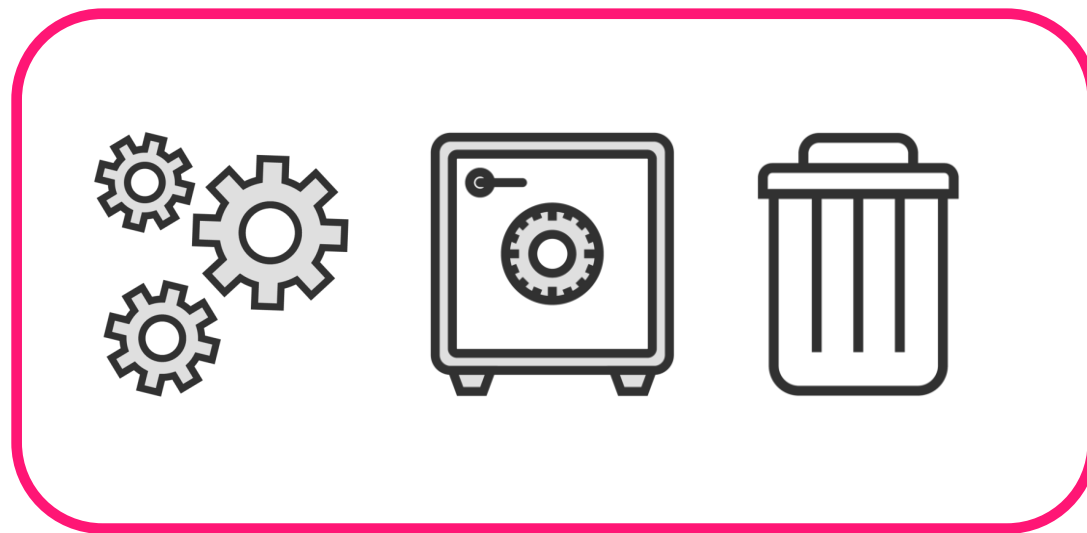
.NET Framework



Common Language Runtimes



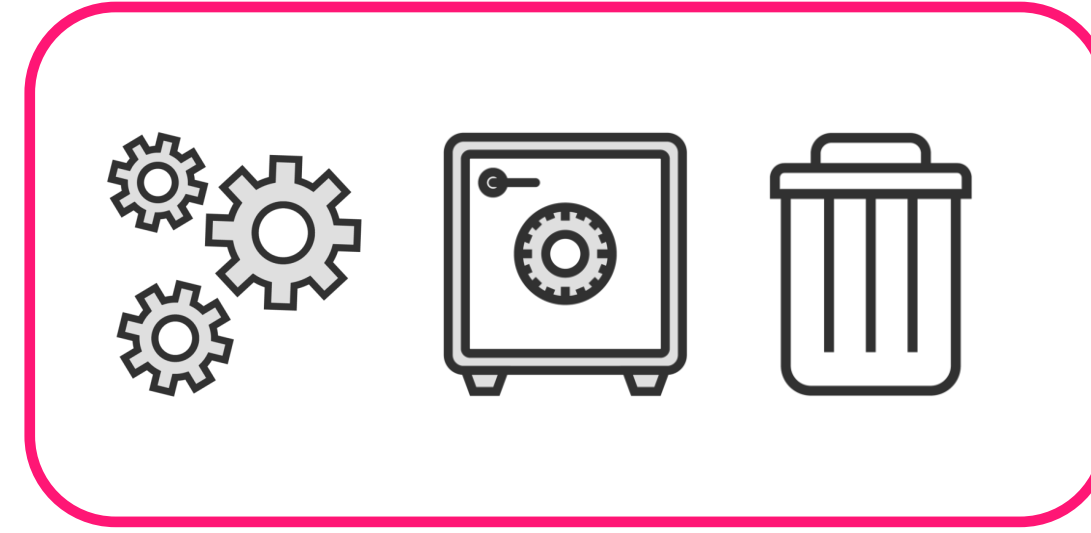
.NET 8



Cross-platform



.NET Framework



Windows

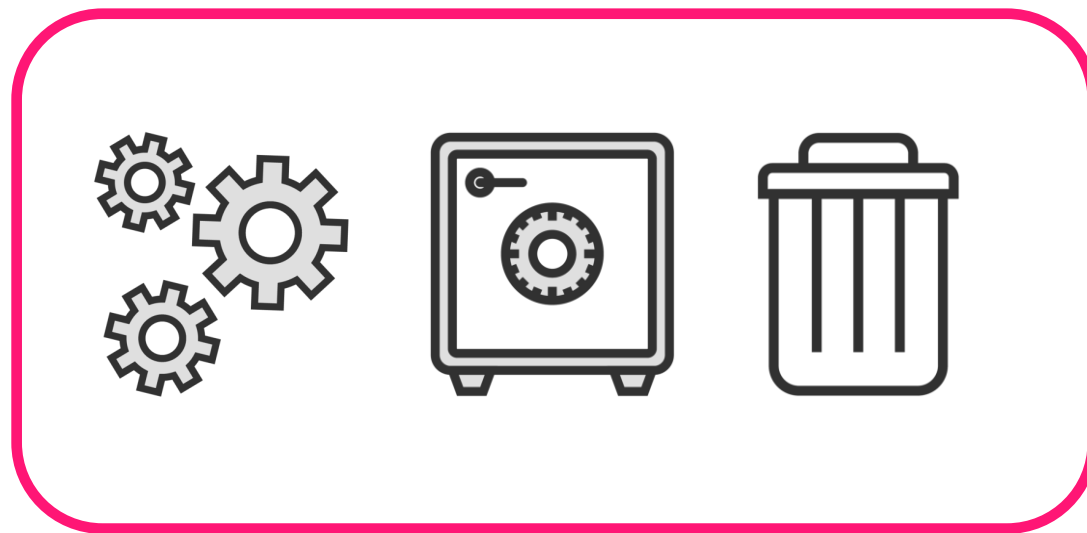


**All new C# projects should
target the cross-platform
version of .NET**



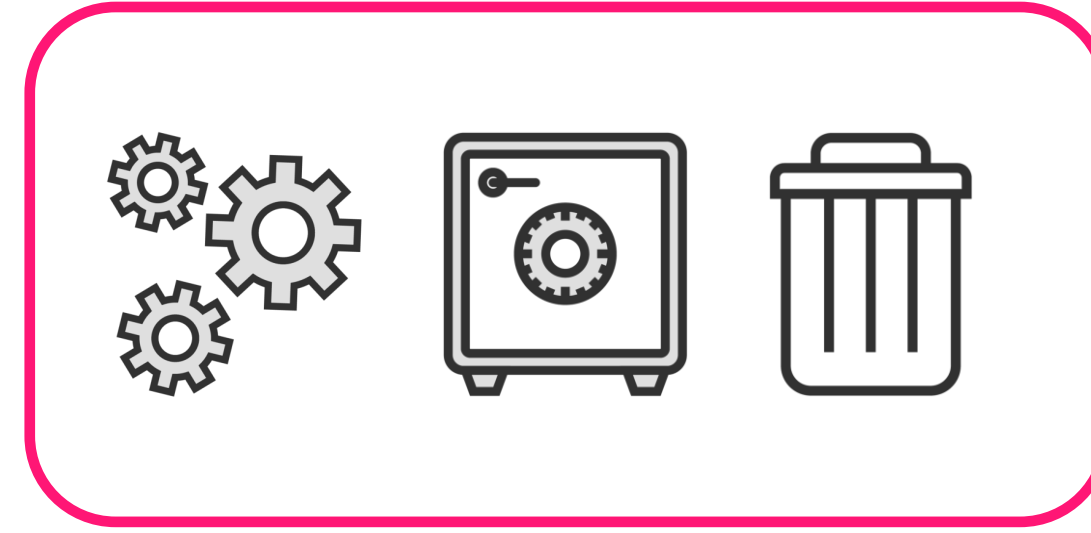
Common Language Runtimes

.NET 8



Cross-platform

.NET Framework



Windows



<https://tinyurl.com/dotnet8platforms>



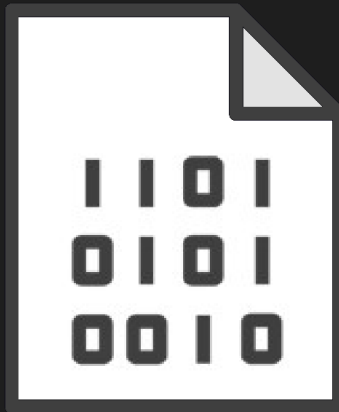


JIT compilation revealed

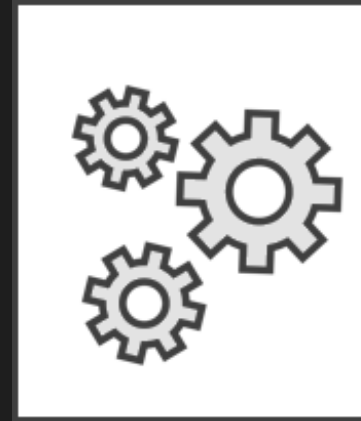
- Proving the native performance claim
- Using specialized tools
- Observing JIT IL-to-machine code generation

Consider this clip OPTIONAL





Main {}

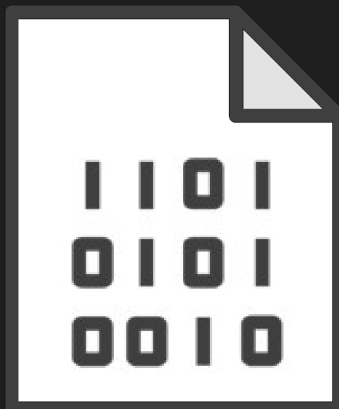


coreclr!PrecodeFixupThunk
(JIT compiler)



Add {}

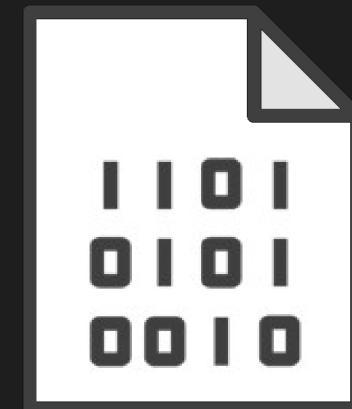
Before JIT Compilation



Main {}



jmp
(zero overhead forward)

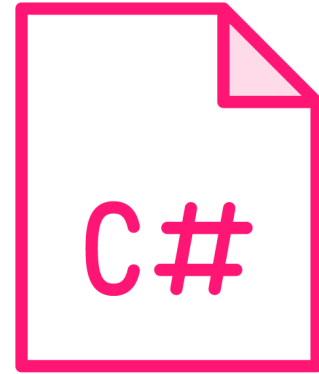


Add {}

After JIT Compilation

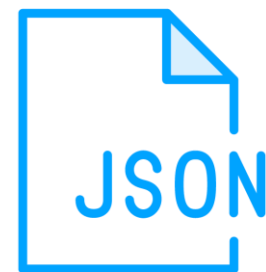


Base Class Libraries (BCL)

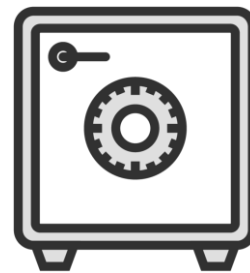


BCL

[A,B,C]



CLR



**By learning C#, you learn
the broader .NET platform**



**By learning .NET, you can
choose the best language
for the task at hand**





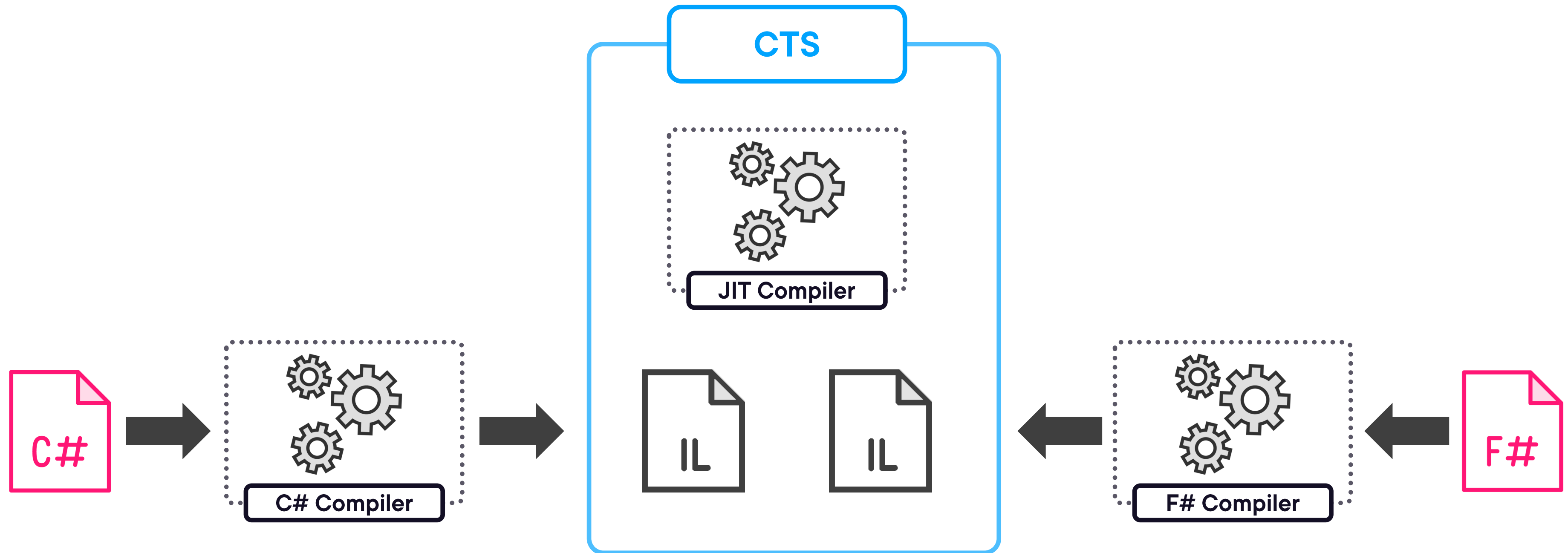
Putting the “CL” in “CLR”

- C# console application
- F# library
- Passing BCL types between them

Consider this clip OPTIONAL



The .NET Common Type System (CTS)



Summary



C# code is compiled into IL assemblies

IL is JIT-compiled at runtime if/when used

JITed code exhibits native performance

The CLR ensures runtime type safety

**The BCL includes general purpose libraries
& app framework functionality**





More information

Class Libraries in C#

Jason Roberts





More information

C# Fundamentals

Gill Cleeren



Up Next:

The Constant Evolution of C#

